

Reviewer Pack — Symmetric Group Multiplicity Gap in Permanent vs Determinant...

Entry 2ad894e61d43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 07:14:24 UTC

Symmetric Group Multiplicity Gap in Permanent vs Determinant Decompositions

Entry ID: 2ad894e61d43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 07:14:24 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality **Field B** (complexity object): algebraic combinatorics

Statement:

For all integers $n \geq 3$, the multiplicity of the Young diagram $\lambda = (n-1, 1)$ in the decomposition of the permanent polynomial under the symmetric group S_n action is strictly greater than its multiplicity in the determinant polynomial's decomposition.

Rationale (proposer's reasoning):

The permanent and determinant have distinct symmetry properties under the symmetric group: the determinant decomposes with non-zero coefficients only for alternating partitions, while the permanent's decomposition includes symmetric partitions. The $(n-1, 1)$ hook shape represents a critical case where symmetric and alternating components intersect. A multiplicity gap here would reflect fundamental asymmetry in their representation-theoretic structures, providing a quantitative barrier for orbit closure containment.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a964b85928adc736

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    if seed > 3:
        return {
            "metric_name": "Multiplicity Gap",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    n = seed + 2
    if n < 3:
        return {
            "metric_name": "Multiplicity Gap",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "n_too_small"
        }

    # Compute Kronecker coefficients for  $\lambda = (n-1, 1)$  in the decomposition of  $\text{Sym}^2(\text{perm})$  vs  $\text{Sym}^2$ 
    def kronecker_coefficient(m, n, k):
        if m == 0 or n == 0 or k == 0:
            return Fraction(1, 1)
        if k > min(m, n):
            return Fraction(0, 1)
        coeff = Fraction(1, 1)
        for i in range(k):
            coeff *= (m - i) * (n - i)
            coeff /= (i + 1) * (k - i)
        return coeff

    def symmetric_group_multiplicity(n, lambda_):
        if len(lambda_) != 2 or lambda_[0] != n-1 or lambda_[1] != 1:
```

```

        return Fraction(0, 1)

    m = lambda_[0]
    k = lambda_[1]

    coeff = kronecker_coefficient(m, m, k) * kronecker_coefficient(m, m, k)
    return coeff

perm_multiplicity = symmetric_group_multiplicity(n, (n-1, 1))
det_multiplicity = symmetric_group_multiplicity(n, (n-1, 1))

# Check if the multiplicity of (n-1,1) in permanent is strictly greater than determinant
conjecture_holds = perm_multiplicity > det_multiplicity

return {
    "metric_name": "Multiplicity Gap",
    "metric_value": perm_multiplicity - det_multiplicity,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "(n-1,1) multiplicity gap is not strictly greater than determinant"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = r["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

instances_tested": 0, "conjecture_holds": False, "counterexample": "mapping_undefined"}
TRIAL: {"seed": 463, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,

```

```

TRIAL: {"seed": 503, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 547, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 593, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 631, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 677, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 727, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 773, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 821, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 877, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
TRIAL: {"seed": 929, "metric_name": "Multiplicity Gap", "metric_value": None, "instances_tested": 0,
RESULT: FALSIFIED counterexample="mapping_undefined" first_failing_seed=929

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

instances_tested remains 0 across all trials, violating the $n \geq 3$ requirement. The metric_value is None for all seeds, indicating the test failed to compute the multiplicity gap. This suggests either a metric definition bug or construction gap preventing proper measurement.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test failed to execute for $n \geq 3$ with metric_value=None across all trials, invalidating the claimed counterexample. | next: Debug metric computation for Young diagram multiplicities in S_n representations

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	52608
2	preregistration	ollama_remote	qwen3:8b	0	0	27766
3	novelty	ollama_remote	qwen3:8b	0	0	24324
4	novelty	ollama_remote	qwen3:8b	0	0	19496
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11551
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7794
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9383
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10504
9	critic	ollama_remote	qwen3:8b	0	0	27811
10	judge	ollama_remote	qwen3:8b	0	0	18283

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 209520 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/2ad894e61d43.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2ad894e61d43.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2ad894e61d43.tar.gz` (if generated)